

Lab 2: Dynamic Programming in Gridworld

CS/DSA 4XX0 – Reinforcement Learning

Overview

In this lab, you will implement the core Dynamic Programming (DP) algorithms used in Reinforcement Learning: **Policy Evaluation**, **Policy Improvement**, **Policy Iteration**, and **Value Iteration**. You will apply these algorithms to a deterministic Gridworld environment.

This lab builds the foundation for later topics such as Monte Carlo methods, Temporal Difference learning, and Q-learning.

All DP algorithms will be implemented in `src/dp.py`. The instructor version is provided in `src/dp_solution.py` for reference and grading.

Learning Objectives

- Understand the Bellman expectation and optimality equations.
- Implement iterative policy evaluation.
- Implement greedy policy improvement.
- Implement full policy iteration.
- Implement value iteration.
- Interpret value functions and policies in Gridworld.

Files Provided

- `src/dp.py` – You implement the DP algorithms here.
- `src/dp_solution.py` – Instructor reference implementation.
- `src/main.py` – Runs your DP algorithms.
- `src/main_solution.py` – Runs instructor algorithms.
- `src/envs/gridworld.py` – Gridworld environment (do not modify).
- `src/utils/utils.py` – Printing utilities (do not modify).
- `tests/test_lab2.py` – Autograder tests.

Gridworld Description

The environment is a 4×4 grid:

- S: Start state
- .: Normal state (reward -1)
- #: Wall (cannot enter)
- T: Trap (reward -10 , terminal)
- G: Goal (reward 10 , terminal)

Actions:

0 = up, 1 = right, 2 = down, 3 = left

All transitions are deterministic.

Your Task

You must implement the following functions in `src/dp.py`:

- `policy_evaluation`
- `policy_improvement`
- `policy_iteration`
- `value_iteration`

Each function currently raises `NotImplementedError`. Replace each `TODO` block with your implementation.

Pseudocode for Required Algorithms

Below is the exact pseudocode you should follow.

Policy Evaluation

Policy Improvement

Policy Iteration

Value Iteration

Algorithm 1 Iterative Policy Evaluation

Require: Policy π , discount γ , threshold θ

```
1: Initialize  $V(s) \leftarrow 0$  for all states
2: repeat
3:    $\Delta \leftarrow 0$ 
4:   for each state  $s$  do
5:      $v \leftarrow V(s)$ 
6:      $V(s) \leftarrow \sum_a \pi(a|s) [R(s, a) + \gamma V(s')]$ 
7:      $\Delta \leftarrow \max(\Delta, |v - V(s)|)$ 
8:   end for
9: until  $\Delta < \theta$ 
10: return  $V$ 
```

Algorithm 2 Greedy Policy Improvement

Require: Value function V , discount γ

```
1: for each state  $s$  do
2:   Compute  $Q(s, a) = R(s, a) + \gamma V(s')$  for all actions  $a$ 
3:   Let  $a = \arg \max_a Q(s, a)$ 
4:   Set  $\pi(s)$  to be deterministic:  $\pi(a|s) = 1$ 
5: end for
6: return  $\pi$ 
```

Running Your Code

From the repository root:

```
python -m src.main
```

This will print:

- Value function from Policy Iteration
- Policy from Policy Iteration
- Value function from Value Iteration
- Policy from Value Iteration

Testing Your Code

Run the autograder:

```
pytest -q
```

You should not modify the test file.

Algorithm 3 Policy Iteration

Require: Discount γ , threshold θ

- 1: Initialize π to be uniform random
 - 2: **repeat**
 - 3: $V \leftarrow \text{PolicyEvaluation}(\pi)$
 - 4: $\pi_{\text{new}} \leftarrow \text{PolicyImprovement}(V)$
 - 5: **until** $\pi_{\text{new}} = \pi$
 - 6: **return** (V, π)
-

Algorithm 4 Value Iteration

Require: Discount γ , threshold θ

- 1: Initialize $V(s) \leftarrow 0$ for all states
 - 2: **repeat**
 - 3: $\Delta \leftarrow 0$
 - 4: **for** each state s **do**
 - 5: $v \leftarrow V(s)$
 - 6: $V(s) \leftarrow \max_a [R(s, a) + \gamma V(s')]$
 - 7: $\Delta \leftarrow \max(\Delta, |v - V(s)|)$
 - 8: **end for**
 - 9: **until** $\Delta < \theta$
 - 10: $\pi \leftarrow$ Greedy policy derived from V
 - 11: **return** (V, π)
-

Submission

Submit the following file:

- `src/dp.py`

Do not modify any other files.

Grading

Your grade is based on:

- Correct implementation of all DP algorithms.
- Passing all autograder tests.
- Producing correct value functions and policies.